

# Predictive Parsing, LL(k) Parsers

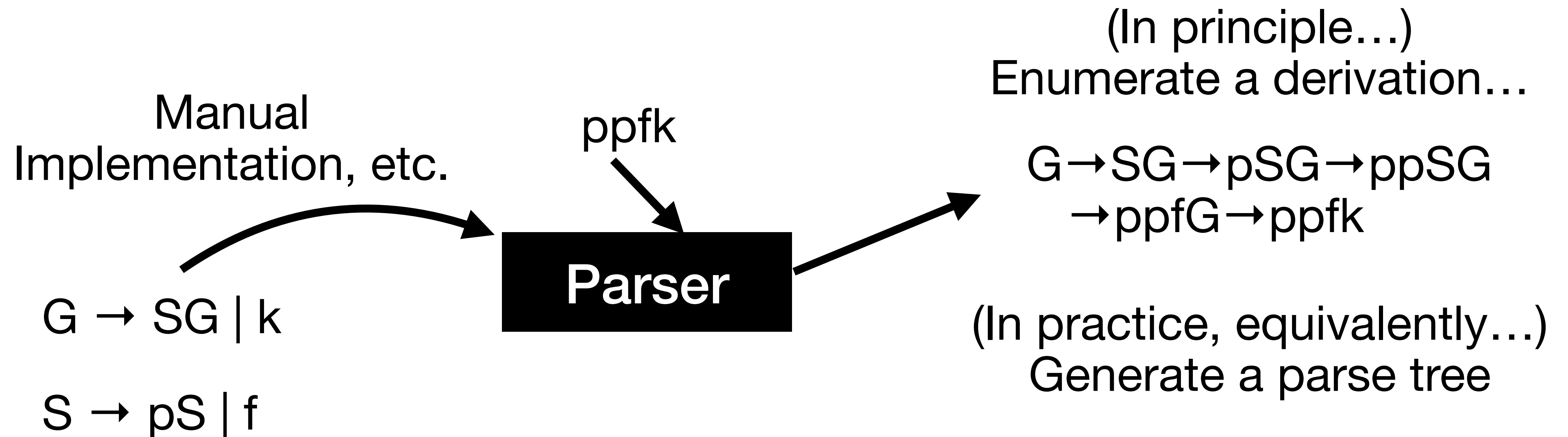
CIS531 — Fall 2025, Syracuse

Kristopher Micinski



# How do we *parse* a given grammar

- Last week: talked about **grammars**, talked about **derivations**
  - A grammar is defined via terminals / nonterminals, start symbol, and productions (rules)
  - Derivations start with the start with the start symbol, follow productions
- This week: given a grammar, how to implement a parser that recognizes it



# If we have a grammar, how do we recognize it...?

- If we start with a given grammar:

$$\begin{aligned} S &\rightarrow "a" S \mid "a" X \\ X &\rightarrow "b" \mid "c" \end{aligned}$$

- The question is how to *recognize* it. We can view this as a search problem (Earley algorithm):
  - Start with the start symbol (S, in this case)
  - Now, consider all possible (branching) derivations, starting from G by running *any* rule
  - This yields a large branching space of possible parse trees
    - When two rules apply—**branch, try both**, generate all possible derivations in parallel
  - 👍 Intuitive, obviously correct
  - 🙌 To recognize “aaaaaaaaaaaaa...ab” we explore  $O(2^n)$  branches before seeing the final b!



# Derivations: Leftmost, Rightmost...

- **Definition:** A given grammar may have any number of derivations for a given string. Among these derivations, we will label two important classes:
  - A **leftmost** derivation is a derivation such that the *expanded* nonterminal is always in the leftmost position, lexically
  - A **rightmost** derivation is a derivation such that the expanded nonterminal is always in the rightmost position, lexically

$$\begin{aligned} S &\rightarrow A B \\ A &\rightarrow "x" A \mid "y" \\ B &\rightarrow "z" B \mid "w" \end{aligned}$$
$$\begin{aligned} S &\rightarrow A B \\ &\rightarrow "x" A B \\ &\rightarrow "x" "x" A B \\ &\rightarrow "x" "x" "y" B \\ &\rightarrow "x" "x" "y" "z" B \\ &\rightarrow "x" "x" "y" "z" "w" \end{aligned}$$
$$\begin{aligned} S &\rightarrow A B \\ &\rightarrow A "z" B \\ &\rightarrow A "z" "w" \\ &\rightarrow "x" A "z" "w" \\ &\rightarrow "x" "x" A "z" "w" \\ &\rightarrow "x" "x" "y" "z" "w" \end{aligned}$$

# Definition: When a grammar is *ambiguous*

- A grammar is **ambiguous** iff there exists some string  $s$  in the grammar such that  $s$  has two *distinct* leftmost derivations (equivalently, two distinct rightmost derivations)
- Below are two grammars—one is ambiguous (this definition), one is not
  - Explain which one is ambiguous—you **must** show multiple leftmost derivations

$$\begin{aligned} S &\rightarrow A B \\ A &\rightarrow "x" \\ B &\rightarrow "y" \end{aligned}$$
$$G \rightarrow G G \mid "a"$$

# Definition: When a grammar is *ambiguous*

- A grammar is **ambiguous** iff there exists some string  $s$  in the grammar such that  $s$  has two *distinct* leftmost derivations (equivalently, two distinct rightmost derivations)
- Below are two grammars—one is ambiguous (this definition), one is not
  - Explain which one is ambiguous—you **must** show multiple leftmost derivations

$$\begin{aligned} S &\rightarrow A B \\ A &\rightarrow "x" \\ B &\rightarrow "y" \end{aligned}$$
$$G \rightarrow G G \mid "a"$$

**Answer: this one, because**

$$\begin{aligned} G &\rightarrow G G \rightarrow "a" G \rightarrow "a" G G \rightarrow "a" "a" G \rightarrow "a" "a" "a" \\ G &\rightarrow G G \rightarrow G G G \rightarrow "a" G G \rightarrow "a" "a" G \rightarrow "a" "a" "a" \end{aligned}$$



# More Practice: Ambiguous Grammars...

$$G \rightarrow G + G$$
$$G \rightarrow G / G$$
$$G \rightarrow \text{number}$$

Draw a leftmost derivation for...

$$1 / 2 / 3$$

Now draw *another* leftmost derivation

# Why is ambiguity important?

- If our grammar is ambiguous there is **no hope to writing a parser** other than one which tries to find a set of derivations, we want derivations to be **unique**
  - Put differently: we don't want to write in a programming language where one string has multiple parse trees—it means that there has to be some policy to disambiguate!
    - Very confusing—in practice, we **disallow** ambiguous grammars for programming
    - However, ambiguous grammars are *common* in natural language
- We will focus on unambiguous grammars



# LL(1), Predictive Parsing

- An LL parser works (a) L—*left-to-right* and produces the (b) L—*leftmost derivation*
- An LL(k) parser is an LL parser that uses k tokens of *lookahead*
- We will expand on these definitions in a few slides, but for now, here is an example of an LL(1) grammar:

$$\begin{aligned} S &\rightarrow A \\ S &\rightarrow BA \\ A &\rightarrow \text{"x"} \\ B &\rightarrow \text{"y"} \end{aligned}$$

- Intuitively, the grammar has the property that we can look at one token of lookahead (e.g., “x” or “y”) and pick which production has to apply
  - If we’re trying to parse S and we see “x”, we know we want  $S \rightarrow A$
  - If we’re trying to parse S and we see “y”, we know we want  $S \rightarrow BA$

# Motivation: FIRST and FOLLOW sets...

- When a grammar is LL(1), then I can look at a single character and *predict* which rule I **must** apply if a parse tree exists—it could fail, but structure of the grammar **commits** us to a choice

$$\begin{array}{l} S \rightarrow A \\ S \rightarrow BA \\ A \rightarrow \text{"x"} \\ B \rightarrow \text{"y"} \end{array}$$

- Consider any leftmost derivation...
  - Begins with...  $S \rightarrow$
  - If I see “x”, then choosing  $S \rightarrow BA$  will **never** work, why?
    - Answer: because B requires me to first match “y”
  - If I see “y”, then choosing  $S \rightarrow A$  will never work, why?
    - Answer: because A requires me to first match “x”

# Definition: FIRST set

- We now present the definitions of the FIRST and FOLLOW sets of a grammar
- Given a grammar  $G$ , consisting of terminals, nonterminals, rules, and a symbol...
- $\text{FIRST}(X)$  is the set of all terminals that can appear as the first symbol of some string derived from  $X$ , where  $X$  is either a nonterminal, a terminal, or a sequence of grammar symbols.
  - If  $X$  is a terminal, then  $\text{FIRST}(X) = \{X\}$ .
  - If  $X$  can derive  $\epsilon$  (the empty string), then  $\epsilon \in \text{FIRST}(X)$ .
  - If  $X$  is a sequence (e.g.,  $YZ\dots$ ), then  $\text{FIRST}(X) \supseteq \text{FIRST}(Y)$  (excluding  $\epsilon$ ), and if  $Y \Rightarrow^* \epsilon$ , then also  $\text{FIRST}(Z)$ , and so on.
- For example, for the grammar:

$S \rightarrow AB$

$A \rightarrow \epsilon \mid a$

$B \rightarrow b$

$\text{FIRST}(A) = \{\epsilon, a\}$

$\text{FIRST}(B) = \{b\}$

$\text{FIRST}(S) = \{a, b\}$



# Definition: FOLLOW set

- For a nonterminal  $A$ ,  $\text{FOLLOW}(A)$  is the set of terminals that can appear *immediately to the right* of  $A$ 
  - If  $S$  is the start symbol then  $\$ \in \text{FOLLOW}(A)$ , where  $\$$  is the end-of-stream symbol
  - For any production that looks like  $B \rightarrow \alpha A a \beta$ , then  $a \in \text{FOLLOW}(A)$
  - For any production that looks like  $B \rightarrow \alpha A X \beta$ , where  $X$  is a nonterminal...
    - $(\text{FIRST}(X) \setminus \{\epsilon\}) \subseteq \text{FOLLOW}(A)$
  - If there is a production  $B \rightarrow \alpha A \beta$  and  $\beta$  is nullable ( $\beta$  can derive  $\epsilon$ ), then...
    - $\text{FOLLOW}(B) \subseteq \text{FOLLOW}(A)$
- We apply these rules iteratively until we learn no more information...

# Example: Calculating a FOLLOW set

- Let's calculate FOLLOW for our tiny grammar...  
 $S \rightarrow A B$   
 $A \rightarrow \varepsilon \mid a$   
 $B \rightarrow b$
- FOLLOW(S): start symbol,  $\{\$ \}$
- FOLLOW(A): in  $S \rightarrow A B$ , we can see that B follows A. So we add FIRST(B), i.e.,  $\{b\}$
- FOLLOW(B): because we have  $S \rightarrow A B$ . Now now that FOLLOW(B) includes FOLLOW(S), so we conclude that  $\text{FOLLOW}(B) = \{\$ \}$
- Thus...  
 $\text{FOLLOW}(S) = \{\$ \}$   
 $\text{FOLLOW}(A) = \{b\}$   
 $\text{FOLLOW}(B) = \{\$ \}$

# Definition: LL(1) Condition

- A grammar is LL(1) iff, for each nonterminal in the grammar  $A$ :
  - For each pair of productions  $A \rightarrow \alpha$  and  $A \rightarrow \beta$  with  $\alpha \neq \beta$ ...
    - $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ 
      - Intuitively: “The FIRST sets of each production of  $A$  are disjoint”
      - Consequence: the first set *uniquely determines* the production
  - If  $\varepsilon \in \text{FIRST}(\alpha)$ , then...
    - $(\text{FIRST}(\alpha) \setminus \{\varepsilon\}) \cap \text{FOLLOW}(A) = \emptyset$
    - Intuitively: No conflict between choosing  $A \rightarrow \varepsilon$  and some other production



# More Practice: FIRST/FOLLOW sets

- Another grammar to try:
- $S \rightarrow A B$   
 $A \rightarrow x \mid y$   
 $B \rightarrow z \mid t$
- $\text{FIRST}(A) = \{x, y\}$ ,  $\text{FIRST}(B) = \{z, t\}$ ,  $\text{FIRST}(S) = \text{FIRST}(A) = \{x, y\}$
- $\text{FOLLOW}(S) = \{\$ \}$ ,  $\text{FOLLOW}(A) = \{z, t\}$ ,  $\text{FOLLOW}(B) = \{\$ \}$
- Does the grammar have the LL1 property?
  - Yes, the first sets for A,B's alternative branches are disjoint!

# Writing the code: Recursive Descent Parsers

- If a grammar is LL(1) we can write a **very simple** parser called a “recursive descent” parser
- Idea: use a function `peek()` to get the *lookahead*
  - Because rules are disjoint, decide which production to take based on lookahead!
  - Each nonterminal *A* turns into a recursive function, `parse_A` which:
    - Branches on the lookahead using `peek()`:
      - Decides which production to apply
      - ***Always possible!*** LL(1) conditions force productions to have *disjoint* START sets
  - When we expect a terminal, call a function `consume('a')` which expects the next character to be 'a' and advances the token stream (error if no match for 'a')



# An example: LL(1) parser

$S \rightarrow A B$   
 $A \rightarrow \epsilon \mid 'a'$   
 $B \rightarrow 'b'$

```
def parse_S():  
    parse_A()  
    parse_B()
```

```
def parse_A():  
    if peek() == 'a':  
        consume('a')  
    return  
else:  
    return
```

```
def parse_B():  
    consume('b')
```

## Example 2: LL(1) parser....

$S \rightarrow 'x' S \mid 'y'$

```
def parse_S():  
    if peek() == 'x':  
        consume('x')  
        # Notice the recursion..  
        parse_S()  
    elif peek() == 'y':  
        consume('y')  
    else:  
        error("parse error, ...")
```

# Example 3

$S \rightarrow id\ Rest$   
 $Rest \rightarrow '( \ Rest \ )' \mid id$

```
def parse_id():  
    # Match current token again  
    # a class of identifiers...
```

```
def parse_S():  
    parse_id()  
    parse_Rest()
```

```
parse_Rest():  
    if peek() == '(':  
        consume('(')  
        parse_Rest()  
        consume(')')  
    else:  
        parse_id()
```



# Writing recursive descent parsers in *Racket*

- In pseudo-code we used `peek()` and `consume()`... but they are very mutable
  - Advance some globalized notion of the “current token”
- In Racket, we want to write ***purely functional*** code, so no possibility of doing this
  - Also, instead of just *matching* as we did in pseudocode, we want to return a tree
- Key idea: each nonterminal *A* turns into a function which accepts the input stream (list)
  - Returns **two** values:
    - (a) the syntax tree (result), along with...
    - (b) the *rest* of the unconsumed token stream

# Issue: Left Recursion

- Recursive descent parsers are simple, intuitive, and generally easy to write
  - 🙅 Unfortunately, *not all grammars are LL(1)*
- One clear issue: LL parsers can **not** handle left recursion:

$$\begin{array}{l} A \rightarrow A - P \mid P \\ P \rightarrow P / I \mid I \\ I \rightarrow \text{number} \end{array}$$

- This grammar *does* have left recursion, and it is helpful in the following way:
- 1 - 2 - 3 should be parsed as (1 - 2) - 3:
  - $A \rightarrow A - P \rightarrow A - P - P \rightarrow P - P - P \rightarrow 1 - P - P \rightarrow 1 - 2 - P \rightarrow 1 - 2 - 3$
  - If we draw the parse tree, we see that we get the intended associativity for -

# LL parsers cannot handle left recursion...

- Grammars with left recursion are *never* LL(k) for any k...

$$\begin{array}{l} A \rightarrow A - P \mid P \\ P \rightarrow P / I \mid I \\ I \rightarrow \text{number} \end{array}$$

- To write `parse_A` we would **immediately recur on A**
  - Yields infinite recursion! Violates LL principles: bounded lookahead predicts production
- So no way to write the above grammar using a recursive descent parser...

```
def parse_A():  
    parse_A() # infinite recursion...  
    consume('-') # never get here...  
    parse_A()
```



# Grammar Transformations: Left Factoring

- In some cases, we can **rewrite** a grammar to be LL(k), one example is *left factoring*
- For example, if we have a rule  $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ ,
- Any grammar including this rule not LL(1): the FIRST sets of both productions include  $\alpha$
- We apply left factoring to split the rule into two rules:  
     $A \rightarrow \alpha A'$   
     $A' \rightarrow \beta_1 \mid \beta_2$
- Common transformation—allows us to make LL(k) parsers LL(1)
  - But in practice: much more natural to write the rules as LL(k)
- Some grammars **cannot** be made LL(k)
  - For these we use hacks (left association) or (more common) use LR parsing
    - LALR, SLR, etc.
    - We will not cover these—but I will demo Yacc a bit

# LR (shift/reduce) parsing

- We did not talk much about the other large class of parsing algorithms, LR parsers
- LR(k) parsers construct the **rightmost** derivation, working left-to-right
  - Nice advantage—no issue with left recursion in grammars!
  - (Handle associativity properly, no factoring/tricks)
- **Key idea:** maintain a stack of symbols (terminals / nonterminals)
  - At every (next) input, you can either shift onto the stack, or reduce the stack by applying a transformation via two tables:
    - *Action table*: shift, reduce, accept, error
    - *Goto table*: jump post-reduction
- 👍 — works for most languages you'd want to write, fast to implement
- 👎 — requires a parser generator (tables are too tedious to do by hand for any nontrivial language), shift/reduce, reduce/reduce conflicts are hard to debug!



# Parsing: Fin

- My goal was to give you the basics of grammars, along with their key properties and transformations. Can you define: grammar, LL(k), LR, recursive descent?
- What to know / practice: could you write a simple recursive-descent parser?
- One exam problem (making clear now): given some relatively simple grammar, can you write a recursive descent parser?
  - You can use any language—if you want to use pseudocode, fine, as long as I can get the idea